ELVARQO / DEVELOPMENT HANDOFF

From design to a working store.

Phase-by-phase execution plan for Codex

REACT STOREFRONT + REACT ADMIN + LARAVEL

Domain: elvarqo.uk
Companion: ELVARQO_Complete_Store_and_Admin_Design.pdf
Plan version 1.0 / 23 October 2026

11 development phases

Each phase contains sequenced build steps, dependencies, design references, completion checks, manual verification and a ready-to-use Codex instruction.

First working milestone

- Create a product in admin, publish it, purchase it using a sandbox payment and find the verified paid order in admin. Deliver this through Phases 1-6.

This document defines proposed work. It does not claim that the website, admin or API has already been implemented.

01 / HOW TO USE THIS PLAN

Give Codex both PDFs.

The design PDF explains appearance. This plan explains behaviour, sequence and completion.

Document contract

- Use the exact companion filename above. Screen IDs S, M, A and D refer to its bookmarks; printed page numbers are also supplied throughout this plan.
- Keep the approved warm ivory, charcoal, olive and sand visual direction. Treat product photos, prices, business metrics and policy copy as concept content until replaced.
- Use this plan for functional requirements and the design PDF for visual layout. If they conflict, identify the conflict and use the latest explicit user instruction.

Execution rhythm

- Start at Phase 1. Break work into the listed substeps, verify each, then complete the phase checks. Continue routine work within the requested phase without unnecessary confirmations.
- After completing the requested phase, give a concise status report and wait for the user to request the next phase. A plan attachment alone is not an instruction to implement all phases at once.
- Inspect the existing repository and preserve unrelated changes. If work already exists, verify it against the criteria and continue from the first genuine gap.

Every phase must leave a usable handoff

- Update the README and a project progress document with completed work, migrations/configuration, design coverage, unresolved issues and next step.
- Run checks appropriate to the changed behaviour and record exact results. Do not claim unrun tests passed; distinguish simulations from real provider integrations.
- Create a focused commit when the repository workflow permits. Never discard local changes, rewrite shared history or commit secrets.

Working assumptions

Premium menswear / UK-first / GBP / guest checkout primary / optional customer accounts / courier shipment booking. Confirm these before dependent work; this current clothing design takes precedence over unrelated earlier concepts.

02 / DELIVERY SEQUENCE

The phase roadmap.

A sequential plan with testable outcomes, rather than a fixed calendar estimate.

Phase	Outcome	Plan pages
1 / Project foundation	Create a reproducible project with a working connection between the React interfaces and Laravel.	5-6
2 / Design system & layouts	Translate the approved visual direction into responsive React components and application shells.	7-8
3 / Admin access & permissions	Provide private staff access before exposing store-management operations.	9-10
4 / Catalogue & inventory management	Let authorised staff publish real variant-based products for the storefront.	11-12
5 / Storefront shopping	Connect the approved shopping interfaces to the real catalogue.	13-14
6 / Bag, checkout & payments	Complete the first end-to-end purchase using sandbox payments and server-calculated totals.	15-16
7 / Orders, fulfilment & delivery bookings	Give staff a reliable workflow from a paid order to tracked dispatch.	17-18
8 / Customer accounts & after-sales	Add optional accounts and traceable returns, exchanges and refunds.	19-20
9 / Content & marketing management	Let staff maintain the storefront without changing code.	21-22
10 / Reporting, support & administration	Complete daily operational tools and visibility across the store.	23-24
11 / Quality, staging & launch	Verify the complete store, rehearse deployment and prepare a controlled production release.	25-26

Three delivery gates

Gate A / after Phase 4: staff can publish a variant product.

Gate B / after Phase 6: a complete sandbox purchase works.

Gate C / after Phase 11: the tested store is prepared for an authorised production release.

03 / TECHNICAL BOUNDARIES

Keep business rules in Laravel.

This is a proposed architecture contract. Verify supported software versions and provider documentation when implementation starts.

Surface	Responsibility
React storefront	Customer browsing, variant selection, guest checkout, optional accounts, content and responsive accessibility. Choose server rendering/pre-rendering for indexable public content in Phase 1.
React admin	Private operational workflows, forms, tables and review steps. Hide unavailable actions for usability, while Laravel independently enforces permission.
Laravel API	Authoritative catalogue, prices, discounts, delivery/tax calculations, inventory reservations, order state, returns, refunds, access control and audit history.
Database & background jobs	Transactional records and constraints; queued mail, imports/exports and integration retries. Store timestamps consistently and display/report in the agreed business timezone.
External adapters	Provider-hosted payments, courier booking/tracking, object storage, transactional email and optional consent-aware marketing. Keep credentials server-side.

Rules that must survive every phase

- Persist money as minor units with currency; never trust prices calculated in the browser. Snapshot ordered descriptions, prices and delivery details.
- Keep payment, fulfilment and return states separate. Verify provider events; make retried financial and stock operations idempotent.
- Add schema incrementally with reviewed migrations. Use a central validation/error format and typed frontend contracts; avoid duplicating business rules in each interface.

PHASE 01 / BUILD SCOPE

Project foundation

Create a reproducible project with a working connection between the React interfaces and Laravel.

DEPENDENCY / Start here. Inspect any existing repository before scaffolding.

DESIGN REFERENCES / Design 00 (p.2), D02-D03 (pp.82-83).

1A - Inspect & record decisions

Read both PDFs and repository guidance. Inventory existing code, tools and local changes. Record launch assumptions and unresolved merchant inputs.

1B - Scaffold the applications

Create a React storefront, React admin and Laravel backend. Use TypeScript for React. Decide server rendering or pre-rendering for public catalogue routes before choosing the frontend framework.

1C - Connect local services

Configure the chosen relational database, storage adapter, queue and email sandbox. Add a health endpoint, environment examples, migrations and repeatable start instructions.

1D - Establish delivery controls

Add formatting, relevant automated checks and a CI workflow. Record selected supported versions, repository structure, API conventions and local development commands.

Data & integration contract

No catalogue or commerce schema is finalised here. Establish migration conventions, API response/error formats, request IDs, timezone policy and a central configuration approach.

Scope boundary

Do not implement product, checkout or dashboard business features yet. Provider credentials are not needed to complete local foundations.

PHASE 01 / VERIFICATION & INSTRUCTION

Prove the phase works.

Completion means the agreed behaviour is working and verified, not just that its screens exist.

Acceptance checks

- A fresh checkout installs and starts using the README and environment examples.
- Storefront and admin can reach the Laravel health endpoint with correctly scoped cross-origin settings.
- Database migrations run on the chosen development database; a queue worker and sandbox email can be exercised.
- No credentials appear in tracked files. Existing uncommitted work remains intact.

Manual verification

- Follow the README from a clean setup and open both React applications.
- Stop the backend, confirm a useful connection error, then restart it and confirm recovery.
- Run the documented checks and record actual results, including any blocked commands.

Give this instruction to Codex

Implement Phase 1 only. Inspect the repository and both PDFs first. Choose a maintainable structure, document supported version choices, connect the React storefront and admin to Laravel, and provide a reproducible local setup. Complete the Phase 1 checks and report decisions, changed files, test results and exact startup instructions.

Required phase report

Summarise implemented behaviour, design screens covered, checks run and results, how to test locally, configuration/migration steps, commit reference if created, unresolved blockers and the next phase. Keep unfinished criteria explicit.

PHASE 02 / BUILD SCOPE

Design system & layouts

Translate the approved visual direction into responsive React components and application shells.

DEPENDENCY / Phase 1 complete.

DESIGN REFERENCES / 00 (p.2); S01-S03 (pp.4-6); M01-M08 (pp.42-49); D01 (p.81).

2A - Extract the visual system

Create shared tokens for charcoal, warm ivory, olive and sand, type styles, spacing, borders and responsive breakpoints. Match the PDF rather than introducing a different theme.

2B - Build reusable controls

Implement buttons, inputs, selects, validation messages, badges, dialogs, drawers, product cards, tables, pagination, loading and empty/error states.

2C - Build application shells

Create the storefront header, mega menu, search overlay, footer and route layout. Create the admin shell with responsive navigation. Use clearly labelled fixtures until APIs exist.

2D - Verify layout and interaction

Compare screenshots against relevant PDF screens. Check keyboard focus, overlay dismissal, touch targets, readable forms and reduced-motion behaviour.

Data & integration contract

Keep fixture data separate from production data access. Define typed UI contracts that can later consume Laravel resources. Add route placeholders only for agreed screens.

Scope boundary

This phase establishes the visual layer. Do not present fixture sales figures or placeholder actions as working store features.

PHASE 02 / VERIFICATION & INSTRUCTION

Prove the phase works.

Completion means the agreed behaviour is working and verified, not just that its screens exist.

Acceptance checks

- Shared components produce consistent desktop and mobile layouts with no unintended horizontal scrolling.
- Keyboard users can open, navigate and close menus/dialogs; focus returns to the trigger.
- Forms have visible labels and useful error presentation; components include loading and disabled states.
- Public pages render usable navigation and meaningful HTML under the chosen rendering strategy.

Manual verification

- Check representative layouts at 390, 768 and 1440 CSS pixels and at 200% zoom.
- Navigate the mega menu and a dialog using only the keyboard.
- Enable reduced motion and confirm decorative transitions are reduced.

Give this instruction to Codex

Implement Phase 2 only using the approved PDF tokens and layouts. Build reusable accessible React components and responsive storefront/admin shells. Keep demonstration data clearly separated. Compare representative desktop and mobile screenshots with the design and report any deliberate deviations.

Required phase report

Summarise implemented behaviour, design screens covered, checks run and results, how to test locally, configuration/migration steps, commit reference if created, unresolved blockers and the next phase. Keep unfinished criteria explicit.

PHASE 03 / BUILD SCOPE

Admin access & permissions

Provide private staff access before exposing store-management operations.

DEPENDENCY / Phases 1-2 complete.

DESIGN REFERENCES / A01 (p.50), A27-A28 (pp.76-77). Roles and audit foundations are built now; management screens are completed in Phase 10.

3A - Define identities & permissions

Keep customer and staff access logically separate. Define permissions for catalogue, fulfilment, customer data, content, finance, staff and integrations.

3B - Build secure staff authentication

Implement login, password recovery, two-step verification, recovery codes, logout and session expiry. Bootstrap the owner account securely without a public staff-registration endpoint.

3C - Enforce access in Laravel

Apply server-side authorisation to protected API operations. Add request validation, throttling and authentication protections appropriate to the selected session architecture.

3D - Establish traceability

Record significant staff security and write events with actor, timestamp, target and result. Build the admin sign-in experience and permission-aware navigation.

Data & integration contract

Introduce staff identities, roles, permissions, recovery/session records and audit events. Never log passwords, recovery secrets, full payment credentials or unnecessary personal data.

Scope boundary

Create only authentication and authorisation foundations. Do not populate unrelated dashboard widgets or build real finance actions yet.

PHASE 03 / VERIFICATION & INSTRUCTION

Prove the phase works.

Completion means the agreed behaviour is working and verified, not just that its screens exist.

Acceptance checks

- Unauthenticated requests are denied; authenticated staff without permission cannot call protected operations directly.
- Two-step verification and recovery work; expired or reused recovery tokens/codes are rejected.
- Logout invalidates the session as designed and staff/customer access cannot be interchanged.
- Relevant access and security events are recorded without exposing secrets.

Manual verification

- Sign in as an owner and exercise two-step verification and logout.
- Use a limited test role and try a finance or staff-management operation through a direct API request.
- Test an expired recovery link and confirm the user receives a safe recovery route.

Give this instruction to Codex

Implement Phase 3 only. Create private staff authentication, two-step verification, server-enforced permissions and audit foundations. Keep customer authentication separate. Test both successful and denied access, including direct API calls, and document how to bootstrap and recover the owner account.

Required phase report

Summarise implemented behaviour, design screens covered, checks run and results, how to test locally, configuration/migration steps, commit reference if created, unresolved blockers and the next phase. Keep unfinished criteria explicit.

PHASE 04 / BUILD SCOPE

Catalogue & inventory management

Let authorised staff publish real variant-based products for the storefront.

DEPENDENCY / Phases 1-3 complete.

DESIGN REFERENCES / A09-A14 (pp.58-63), A20 (p.69), A31 (p.80). S08-S10 (pp.11-13) define the information each product must supply.

4A - Model products & variants

Create collections/categories, products, options, variants, unique SKUs, prices and publication states. Link images, measurements, fit and care information to the appropriate product or variant.

4B - Build staff catalogue tools

Implement product lists, draft editing, preview, publication, category ordering, variant editing and image upload/reordering with alternative text.

4C - Establish stock accounting

Store warehouse/location balances and traceable stock movements. Separate on-hand, reserved and available quantities; reservation transactions are completed in Phase 6.

4D - Expose the public catalogue

Add validated read endpoints exposing only published products and active variants. Build media validation and responsive image delivery; include usable product SEO fields.

Data & integration contract

Core records: products, categories/collections, option values, variants, product media, prices, stock locations, movements and reservation capacity. Store money using integer minor units plus currency.

Scope boundary

No fake product claims or inventory in production. General CSV import/export tooling remains in Phase 10; checkout stock reservation follows in Phase 6.

PHASE 04 / VERIFICATION & INSTRUCTION

Prove the phase works.

Completion means the agreed behaviour is working and verified, not just that its screens exist.

Acceptance checks

- Staff can create a product with colour/size variants, publish it and retrieve it through the public catalogue API.
- Draft products stay private, duplicate SKUs are rejected and invalid price/media inputs produce clear errors.
- Stock adjustments require a reason and preserve a traceable movement history.
- Product and media deletion preserve historical order integrity; published slugs have defined redirect behaviour.

Manual verification

- Create a polo with two colours and two sizes, upload images and publish it.
- Try duplicate SKUs, invalid images and a negative selling price.
- Record incoming stock, inspect the movement history and unpublish the product.

Give this instruction to Codex

Implement Phase 4 only. Build the catalogue, variants, media, publication and inventory foundations with protected admin tools and public read APIs. Use the referenced design screens. Verify a full create-preview-publish-unpublish journey and demonstrate SKU, price and stock validation.

Required phase report

Summarise implemented behaviour, design screens covered, checks run and results, how to test locally, configuration/migration steps, commit reference if created, unresolved blockers and the next phase. Keep unfinished criteria explicit.

PHASE 05 / BUILD SCOPE

Storefront shopping

Connect the approved shopping interfaces to the real catalogue.

DEPENDENCY / Phases 1-4 complete.

DESIGN REFERENCES / S01-S12 (pp.4-15), S38 (p.41), M01-M03 (pp.42-44).

5A - Implement product discovery

Connect homepage product sections, collections, listings and search to Laravel. Persist filters, sort and pagination in URLs with stable result counts.

5B - Complete product detail

Build the gallery, colour/size selection, price, stock, size guide, material and care sections. Correctly link variants to their images and availability.

5C - Add saved items & selection flows

Implement a guest wishlist in local storage and the quick-selection interface. Define the bag handoff contract; real add-to-bag processing arrives in Phase 6.

5D - Finish responsive recovery states

Handle empty searches, unavailable products, missing routes, request failures and image loading. Apply route metadata, canonical rules and the selected public-page rendering strategy.

Data & integration contract

Use published catalogue resources as the source of truth. Do not hard-code stock or prices in React. Catalogue search must exclude draft products. Signed-in wishlist syncing follows in Phase 8.

Scope boundary

Do not invent verified reviews or product certifications. Basic homepage merchandising can use configuration until the visual CMS is built in Phase 9.

PHASE 05 / VERIFICATION & INSTRUCTION

Prove the phase works.

Completion means the agreed behaviour is working and verified, not just that its screens exist.

Acceptance checks

- Listings, filters, search and product detail display server data, including current published prices and stock.
- Filter URLs reload consistently and zero-result searches offer a useful recovery action.
- Variant selection and size guides remain usable on mobile, keyboard and screen readers.
- Product routes return appropriate metadata and missing/unpublished routes have an intentional response.

Manual verification

- Find a product using category browsing, search and a filter URL opened in a fresh tab.
- Select each variant, inspect unavailable sizes and reopen the saved wishlist.
- Check mobile layout and temporarily fail the catalogue request to confirm retry behaviour.

Give this instruction to Codex

Implement Phase 5 only. Connect the storefront designs to the real catalogue, including discovery, filters, variant details, size guides and guest wishlist. Keep prices and availability server-backed. Implement recovery states and document the handoff to the bag work in Phase 6.

Required phase report

Summarise implemented behaviour, design screens covered, checks run and results, how to test locally, configuration/migration steps, commit reference if created, unresolved blockers and the next phase. Keep unfinished criteria explicit.

PHASE 06 / BUILD SCOPE

Bag, checkout & payments

Complete the first end-to-end purchase using sandbox payments and server-calculated totals.

DEPENDENCY / Phases 1-5 complete. Confirm payment provider, launch market, tax approach and delivery rules before configuring real transactions.

DESIGN REFERENCES / S13-S18 (pp.16-21), M04-M06 (pp.45-47); minimum A18, A22, A25-A26, A29 settings (pp.67,71,74-75,78).

6A - Build bag & pricing

Implement a guest bag and server-side pricing for variants, quantities, valid discount codes, shipping and applicable tax. Display final totals before payment.

6B - Create checkout transactions

Validate addresses and available delivery methods. Reserve stock atomically with an expiry policy. Persist immutable order item/address/price snapshots and separate order, payment and fulfilment states.

6C - Integrate sandbox payments

Use hosted payment fields or checkout. Create payment attempts with idempotency, verify webhook signatures and handle duplicate, late and out-of-order events safely.

6D - Complete confirmation & recovery

Confirm paid status only from trusted provider evidence. Show pending/failure/retry states, release expired reservations and queue transaction-safe order emails. Add a protected read-only order view in admin; full operations follow in Phase 7.

Data & integration contract

Add bags, checkout attempts, orders/items, address snapshots, stock reservations, payment attempts, webhook events and a minimal discount/rate engine. Basic rate and provider configuration is needed now; richer editors follow later.

Scope boundary

Sandbox-only until release approval. Do not mark simulated provider responses as a working live payment integration. Never collect raw card data in Laravel.

PHASE 06 / VERIFICATION & INSTRUCTION

Prove the phase works.

Completion means the agreed behaviour is working and verified, not just that its screens exist.

Acceptance checks

- A successful sandbox purchase creates exactly one correctly priced order, visible in a protected read-only admin view.
- Browser-modified prices are ignored and two simultaneous checkouts cannot oversell the same remaining unit.
- Duplicate webhooks or repeated submissions do not duplicate payments, orders, stock deductions or emails.
- Failed/expired payment attempts preserve a recovery path and release stock according to policy; redirects alone cannot mark an order paid.

Manual verification

- Purchase a variant as a guest; verify the payment, order snapshot and reserved/committed stock.
- Exercise decline, pending, abandoned checkout, retry and webhook replay scenarios.
- Run concurrent purchases for a last unit and confirm that only eligible stock is sold.

Give this instruction to Codex

Implement Phase 6 only, in substeps 6A-6D. Make Laravel authoritative for money, inventory and order state. Complete a sandbox guest purchase with hosted payment capture, verified idempotent webhooks, stock reservations and recoverable failures. Prove duplicate and concurrent requests do not produce duplicate orders or overselling.

Required phase report

Summarise implemented behaviour, design screens covered, checks run and results, how to test locally, configuration/migration steps, commit reference if created, unresolved blockers and the next phase. Keep unfinished criteria explicit.

PHASE 07 / BUILD SCOPE

Orders, fulfilment & delivery bookings

Give staff a reliable workflow from a paid order to tracked dispatch.

DEPENDENCY / Phase 6 complete. Courier selection is required for a real label integration; a clearly labelled adapter sandbox can be used first.

DESIGN REFERENCES / A03-A06 (pp.52-55), A25 (p.74), relevant A29 (p.78), S19 (p.22), M08 (p.49).

7A - Build order operations

Implement order search, filters, item detail, internal notes, addresses and payment/fulfilment state display. Support manual orders through the same pricing and stock services.

7B - Build dispatch & courier booking

Select items/quantities for fulfilment, validate parcel data, get service quotes and review label purchase. Record booking attempts, provider references, label files and tracking numbers.

7C - Support fulfilment exceptions

Handle partial shipments, booking failures, safe retries and cancellation eligibility. Prevent duplicate purchased labels and invalid state transitions.

7D - Expose customer progress

Build secure guest tracking with verified access, shipment status ingestion and dispatch notifications. Enable a minimal order-support request route to be expanded in Phase 10.

Data & integration contract

Add fulfilments/items, parcels, booking attempts, labels, tracking events and notification records. Courier booking means shipment booking; customer appointment booking is outside the agreed retail scope.

Scope boundary

No unrequested appointment/rental booking system. Manual orders use secure payment links or an explicitly authorised payment-recording process.

PHASE 07 / VERIFICATION & INSTRUCTION

Prove the phase works.

Completion means the agreed behaviour is working and verified, not just that its screens exist.

Acceptance checks

- Staff can dispatch a paid order, obtain a sandbox/provider label and expose accurate tracking to the customer.
- Partial fulfilment never exceeds ordered quantities; repeated booking attempts do not silently buy extra labels.
- Webhook events are verified, deduplicated and mapped to shipment state without overwriting unrelated order state.
- Guests cannot see order addresses or shipment details without the required verified tracking access.

Manual verification

- Fulfil an order in two shipments and check remaining quantities.
- Simulate courier timeout, retry safely and inspect the booking history.
- Request a guest tracking link; try another customer's order reference without verified access.

Give this instruction to Codex

Implement Phase 7 only. Build protected order management, manual orders, partial fulfilment, courier shipment booking, labels and secure customer tracking. Reuse checkout pricing and stock logic. Show booking failure/retry behaviour, traceable notifications and proof that repeated requests do not purchase duplicate labels.

Required phase report

Summarise implemented behaviour, design screens covered, checks run and results, how to test locally, configuration/migration steps, commit reference if created, unresolved blockers and the next phase. Keep unfinished criteria explicit.

PHASE 08 / BUILD SCOPE

Customer accounts & after-sales

Add optional accounts and traceable returns, exchanges and refunds.

DEPENDENCY / Phases 6-7 complete, with Phase 3 permission foundations.

DESIGN REFERENCES / S20-S27 (pp.23-30), A07-A08 (pp.56-57), M07 (p.48).

8A - Add optional customer accounts

Implement registration, email verification, login/recovery, logout, profile and address editing. Preserve guest checkout as the primary purchase route.

8B - Connect customer records

Show customer-owned order history/detail and saved items. Associate guest orders only after verified ownership; merge guest wishlists without losing entries.

8C - Build return & exchange handling

Create item-level requests, eligibility rules, reasons, inspection outcomes and return timelines. Model exchange stock and any price/payment difference explicitly.

8D - Implement controlled refunds

Review refundable balances, require the relevant staff permission and confirm amount/reason. Send provider refund requests idempotently and reconcile asynchronous outcomes.

Data & integration contract

Add customer identities/verification, account addresses, saved products, return requests/items, inspections, exchange links and refund attempts. Preserve historical order addresses when a customer edits their current address.

Scope boundary

Policies and eligibility values must match merchant-approved content. Refunds must not be recorded as completed until the provider outcome supports that state.

PHASE 08 / VERIFICATION & INSTRUCTION

Prove the phase works.

Completion means the agreed behaviour is working and verified, not just that its screens exist.

Acceptance checks

- A customer can access only their own account, wishlist, addresses and orders; staff sessions cannot be substituted.
- Guest orders are linked only after ownership verification, not because a typed email happens to match.
- Return quantities and refund amounts cannot exceed the eligible remaining balances.
- Provider refund retries are idempotent; failed/pending refunds remain visible and stock is restored only after the correct inspection outcome.

Manual verification

- Create two customer accounts and attempt cross-account order/address access.
- Request a partial return, inspect it and complete a sandbox partial refund.
- Try refund replay and an unavailable exchange size; confirm safe recovery and a traceable history.

Give this instruction to Codex

Implement Phase 8 only. Build optional customer accounts, ownership-protected order history, addresses and wishlist syncing. Add item-level returns, exchanges, inspection and permission-controlled idempotent refunds. Test cross-account access denial, partial returns and failed or repeated refund attempts.

Required phase report

Summarise implemented behaviour, design screens covered, checks run and results, how to test locally, configuration/migration steps, commit reference if created, unresolved blockers and the next phase. Keep unfinished criteria explicit.

PHASE 09 / BUILD SCOPE

Content & marketing management

Let staff maintain the storefront without changing code.

DEPENDENCY / Phases 4-8 complete. Reuse the discount engine established in Phase 6.

DESIGN REFERENCES / S28-S37 (pp.31-40), A17-A21 (pp.66-70), A31 (p.80). Basic support requests here connect to the Phase 10 inbox.

9A - Build homepage & navigation editing

Create reusable section configuration for hero, featured collections/products, brand content and newsletter. Add draft, preview, publish and revision rollback.

9B - Build pages & journal

Implement rich content editing, media usage, slugs, metadata and publication. Publish approved help, delivery, returns, privacy, terms and contact content.

9C - Complete promotions

Build discount configuration, eligibility/exclusions, usage limits, date scheduling and coupon feedback. Keep calculations in the shared server pricing service.

9D - Add consent-aware marketing

Implement optional newsletter consent and configurable transactional/marketing templates. Keep optional tracking disabled until the visitor permits it; provide preference changes.

Data & integration contract

Add content pages/revisions, journal articles, navigation, homepage sections, promotion schedules, newsletter preferences and consent records. Sanitize rich HTML and validate media, links and publication times.

Scope boundary

Use genuine product and business copy. Do not publish generated concept photographs as evidence of actual stock, or invent reviews, sourcing claims or discount urgency.

PHASE 09 / VERIFICATION & INSTRUCTION

Prove the phase works.

Completion means the agreed behaviour is working and verified, not just that its screens exist.

Acceptance checks

- A staff member previews and publishes a banner/page, then restores a prior revision.
- Stored rich content cannot execute injected script; public routes have correct titles, canonical rules and sitemap inclusion.
- Promotion expiry, exclusions and usage limits agree between admin preview and checkout, including concurrent redemptions.
- Essential-only consent prevents optional tracking, and marketing opt-out is respected across integrations.

Manual verification

- Publish a homepage change and journal article, then confirm the public page and metadata.
- Test one eligible and one excluded discount purchase, including an expired code.
- Choose essential-only cookies, inspect optional requests, then change preferences.

Give this instruction to Codex

Implement Phase 9 only. Create a safe revisioned CMS for homepage, navigation, pages and journal, then complete promotion management and consent-aware marketing. Reuse existing media and pricing services. Verify publication/rollback, rich-content safety, discount eligibility and essential-only tracking behaviour.

Required phase report

Summarise implemented behaviour, design screens covered, checks run and results, how to test locally, configuration/migration steps, commit reference if created, unresolved blockers and the next phase. Keep unfinished criteria explicit.

PHASE 10 / BUILD SCOPE

Reporting, support & administration

Complete daily operational tools and visibility across the store.

DEPENDENCY / Phases 3-9 complete. Use real operational records, not dashboard fixtures.

DESIGN REFERENCES / A02 (p.51), A15-A16 (pp.64-65), A22-A24 (pp.71-73), A26-A30 (pp.75-79), S32 (p.35).

10A - Build financial & stock reporting

Replace overview fixtures with actual order data. Define gross/net sales, discounts, refunds, average order value, stock warnings and reporting periods. Add payment reconciliation and exportable reports.

10B - Complete support & customer records

Build the support inbox, assignment, status, linked orders and permission-limited customer detail. Use an explicit review action for outgoing replies and log the result.

10C - Complete staff & store settings

Build invitations, role editing, integration health, notification templates and audit-log views. Prevent removal of the last usable owner and protect secret configuration.

10D - Build operational jobs

Implement CSV templates, import preview/validation, queued execution, per-row errors, authorised exports and job history. Complete the planned stock-transfer workflow with origin/destination movements.

Data & integration contract

Aggregate authoritative order/payment/refund records; define timezone boundaries and refund timing for reports. Add support threads, staff invitations, import/export jobs and a balanced stock-transfer record.

Scope boundary

Do not add a separate accounting, ERP or appointment platform. Reports must state their definitions and distinguish test/sandbox records from real business figures.

PHASE 10 / VERIFICATION & INSTRUCTION

Prove the phase works.

Completion means the agreed behaviour is working and verified, not just that its screens exist.

Acceptance checks

- Report totals reconcile with a known set of orders, discounts, delivery and partial refunds; date ranges have consistent boundaries.
- Staff roles limit customer data, exports, support actions and refund access; secret settings stay masked and out of logs.
- Invalid imports show row-level reasons; retries do not duplicate records and exports cannot leak another role's restricted data.
- Queue failures, integration outages and support delivery failures remain visible with an actionable retry path.

Manual verification

- Compare a fixed order/refund fixture set with dashboard totals and an exported report.
- Import a CSV with valid and invalid rows, then retry the corrected file.
- Invite a limited staff user, process a support ticket and inspect the resulting audit events.

Give this instruction to Codex

Implement Phase 10 only. Replace dashboard fixtures with reconciled operational reports and finish support, customer records, staff/settings, audit views and import/export jobs. Reuse existing permissions and finance logic. Prove reporting totals, export access control, row-level import validation and recoverable job failures.

Required phase report

Summarise implemented behaviour, design screens covered, checks run and results, how to test locally, configuration/migration steps, commit reference if created, unresolved blockers and the next phase. Keep unfinished criteria explicit.

PHASE 11 / BUILD SCOPE

Quality, staging & launch

Verify the complete store, rehearse deployment and prepare a controlled production release.

DEPENDENCY / Phases 1-10 complete. Real catalogue, policies, provider accounts and fulfilment details must be supplied before launch.

DESIGN REFERENCES / All S01-S38, M01-M08 and A01-A31 screens. D01-D03 (pp.81-83) define cross-cutting expectations.

11A - Close the implementation gaps

Audit every referenced screen, route and action against the design and this roadmap. Complete mobile states, validation, empty/loading states and production content.

11B - Verify risky journeys

Exercise guest/account orders, concurrent stock claims, payment retries, webhook replays, partial dispatch, returns, exchanges, refunds and permission boundaries.

11C - Rehearse staging operations

Configure staging, migrations, worker scheduling, mail/storage, provider callbacks and monitoring. Perform a backup restore drill and document an application/data rollback approach.

11D - Prepare production release

Confirm domain/TLS, environment isolation, credentials, transactional mail, assets, policies, stock and merchant-approved settings. Prepare a release checklist, smoke test and support runbook.

Data & integration contract

Keep development, staging and production credentials/data isolated. Use additive migrations where practical, review data changes separately and prohibit destructive test resets in production.

Scope boundary

Do all preparation before requesting any missing release authorisation. Do not claim production readiness when provider, content, stock or recovery checks remain unverified.

PHASE 11 / VERIFICATION & INSTRUCTION

Prove the phase works.

Completion means the agreed behaviour is working and verified, not just that its screens exist.

Acceptance checks

- All agreed screens/actions are implemented or explicitly recorded as deferred with user agreement; no misleading fixture metrics or broken controls remain.
- Automated checks and representative browser/device tests pass; accessibility and performance findings are documented and significant issues resolved.
- A backup can be restored, queues/webhooks are observable and deployment/rollback commands have been rehearsed.
- Production launch readiness is evidenced; actual publication, live charges or production data operations follow the user's explicit release instruction.

Manual verification

- Run a full sandbox order-to-shipment-to-return-to-refund rehearsal in staging.
- Rebuild from a clean checkout and restore a backup to an isolated environment.
- Use the merchant-approved release checklist, then conduct a small controlled live transaction only when authorised.

Give this instruction to Codex

Implement Phase 11 only. Audit all design screens, close functional gaps and verify complete commerce journeys in staging. Rehearse deployment, recovery and observability. Deliver a concrete launch checklist with evidence, unresolved blockers and rollback instructions; do not publish or initiate live transactions without the user's release instruction.

Required phase report

Summarise implemented behaviour, design screens covered, checks run and results, how to test locally, configuration/migration steps, commit reference if created, unresolved blockers and the next phase. Keep unfinished criteria explicit.

04 / RELEASE INPUTS

Decisions and real content.

These inputs must be confirmed before the dependent feature can be treated as live-ready.

Input	Needed by	Required decision
Launch range	Phase 1 / real data in 4	Actual categories, suppliers, variant options, stock, prices and verified product information.
Market & business	Phase 1; checkout in 6	Selling countries, base currency, business details and merchant-confirmed tax/invoicing approach.
Brand assets	Phase 2; final in 11	Approved logo/fonts and accurate product/campaign photography. Generated concept photos are not product evidence.
Payment provider	Phase 6	Merchant account, sandbox credentials, supported methods, webhook setup and production activation.
Delivery & booking	Rates in 6; labels in 7	Origin warehouse, zones, courier/service choices, parcel data, actual rates and tracking behaviour.
Return rules	Phase 8; publish in 9	Actual eligibility, exclusions, costs, inspection process, exchange and refund rules.
Email & content	Sandbox in 1; live by 11	Sending domain, approved templates, real support contact details and reviewed policy copy.
Hosting & recovery	Plan in 1; rehearse in 11	Deployment targets, database, storage, queue/scheduler, backups, monitoring and release ownership.

If an input is missing, implement a provider-neutral interface or a clearly labelled sandbox where practical. Continue independent work; ask only for decisions that block the next dependent action.

05 / DESIGN COVERAGE

Screen ownership by phase.

This table covers every S, M and A design screen. Revisit shared screens as their backend capabilities arrive.

Phase	Primary design ownership
1-2 / Foundations	Design 00 and D01-D03. Shared shells and controls across all screens; no claim of functional completion.
3 / Staff access	A01. Permission/audit foundations for A27-A28.
4 / Catalogue	A09-A14, A20, A31. Product data requirements from S08-S10.
5 / Shopping	S01-S12 and S38; M01-M03. Shared storefront navigation/footer.
6 / Checkout	S13-S18; M04-M06. Minimum provider/rate/discount settings backing A18, A22, A25-A26, A29.
7 / Fulfilment	A03-A06, A25 and courier parts of A29; S19; M08.
8 / Accounts & returns	S20-S27; A07-A08; M07. Signed-in wishlist behaviour extends S12.
9 / Content & marketing	S28-S37; A17-A21 and content extensions to A31. S32 form connects to support.
10 / Operations	A02, A15-A16, A22-A24, A26-A30; S32 support completion. Finish stock transfers from A14.
11 / Release	Audit and verify all S01-S38, M01-M08 and A01-A31; resolve D01-D03 requirements.

Design clarifications for implementation

- The S06 filter illustration shows a draft filter selection beside existing results. Implement a consistent draft/applied filter state and matching result count; do not reproduce inconsistent sample counts.
- Admin charts and tables are sample layouts. Derive production totals from real records and document metric definitions.
- The PDF depicts representative mobile views. All storefront and operational routes still need responsive behaviour, even where no separate mobile illustration exists.

06 / START HERE

Your first message to Codex.

Attach this plan and the companion design PDF, then paste the instruction below.

Copy into Codex

We are building ELVARQO for elvarqo.uk using a React storefront, React admin dashboard and Laravel backend. Read both attached PDFs: ELVARQO_Complete_Store_and_Admin_Design.pdf and ELVARQO_Phase_by_Phase_Codex_Development_Plan.pdf.

Use the design PDF for visual appearance and the development plan for behaviour, dependencies and completion checks. Start with Phase 1 only. First inspect the existing repository, local changes and project guidance. If the repository is empty, scaffold the agreed application structure; if code exists, preserve it and continue from verified gaps.

Document architecture and version decisions, configure a reproducible local setup, connect both React interfaces to Laravel and complete the Phase 1 acceptance checks. Do not build unrelated phases or change the approved visual direction. Record assumptions; request clarification only when a missing decision blocks dependent work.

Update the README and progress document. Finish with implemented work, files changed, exact commands and test results, startup/manual test steps, configuration requirements, unresolved issues and a focused commit if appropriate. Stop after completing and reporting Phase 1; I will request Phase 2 next.

For later phases

- Use the phase-specific instruction included on each verification page. Keep attaching or retaining both documents in the project context.
- Say: "Continue with Phase [number] only. Read the latest progress report, verify prerequisites and follow the acceptance checks in the plan."
- If a check fails, finish the fixes within that phase before presenting it as completed. Do not carry hidden failures into the next phase.

Source alignment

Prepared against the 85-page ELVARQO visual design dated 23 October 2026 and the agreed eleven-phase roadmap. All design references use its existing screen IDs and printed page numbers.